

# Queue Problems

---

## ✓ 1. 933. Number of Recent Calls

```
from collections import deque
class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

## ✓ 2. 346. Moving Average from Data Stream

```
from collections import deque
class MovingAverage:
    def __init__(self, size: int):
        self.q = deque()
        self.size = size
        self.total = 0

    def next(self, val: int) -> float:
        self.q.append(val)
        self.total += val
        if len(self.q) > self.size:
            self.total -= self.q.popleft()
        return self.total / len(self.q)
```

## ✓ 3. 641. Design Circular Deque

```
class MyCircularDeque:
    def __init__(self, k: int):
        self.q = []
        self.k = k
```

```

def insertFront(self, value: int) -> bool:
    if len(self.q) < self.k:
        self.q.insert(0, value)
        return True
    return False

def insertLast(self, value: int) -> bool:
    if len(self.q) < self.k:
        self.q.append(value)
        return True
    return False

def deleteFront(self) -> bool:
    if self.q:
        self.q.pop(0)
        return True
    return False

def deleteLast(self) -> bool:
    if self.q:
        self.q.pop()
        return True
    return False

def getFront(self) -> int:
    return self.q[0] if self.q else -1

def getRear(self) -> int:
    return self.q[-1] if self.q else -1

def isEmpty(self) -> bool:
    return not self.q

def isFull(self) -> bool:
    return len(self.q) == self.k

```

## ✓ 4. 622. Design Circular Queue

```

class MyCircularQueue:
    def __init__(self, k: int):
        self.q = [None] * k
        self.head = self.tail = -1
        self.k = k

    def enqueue(self, value: int) -> bool:
        if self.isFull():

```

```

        return False
    if self.isEmpty():
        self.head = self.tail = 0
    else:
        self.tail = (self.tail + 1) % self.k
    self.q[self.tail] = value
    return True

def deQueue(self) -> bool:
    if self.isEmpty():
        return False
    if self.head == self.tail:
        self.head = self.tail = -1
    else:
        self.head = (self.head + 1) % self.k
    return True

def Front(self) -> int:
    return -1 if self.isEmpty() else self.q[self.head]

def Rear(self) -> int:
    return -1 if self.isEmpty() else self.q[self.tail]

def isEmpty(self) -> bool:
    return self.head == -1

def isFull(self) -> bool:
    return (self.tail + 1) % self.k == self.head

```

## 5. 1700. Number of Students Unable to Eat Lunch

```

from collections import deque
class Solution:
    def countStudents(self, students: list[int], sandwiches: list[int]) -> int:
        students = deque(students)
        sandwiches = deque(sandwiches)
        attempts = 0

        while students:
            if students[0] == sandwiches[0]:
                students.popleft()
                sandwiches.popleft()
                attempts = 0
            else:
                students.append(students.popleft())
                attempts += 1

```

```

        if attempts == len(students):
            break
    return len(students)

```

## ✓ 6. 1425. Constrained Subsequence Sum

```

from collections import deque
class Solution:
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
        dp = nums[:]
        q = deque()
        for i in range(len(nums)):
            if q:
                dp[i] = max(dp[i], nums[i] + dp[q[0]])
                while q and dp[i] >= dp[q[-1]]:
                    q.pop()
            if dp[i] > 0:
                q.append(i)
            if i - q[0] >= k:
                q.popleft()
        return max(dp)

```

## ✓ 7. 170. Two Sum III – Data structure design

```

from collections import defaultdict
class TwoSum:
    def __init__(self):
        self.freq = defaultdict(int)

    def add(self, number: int) -> None:
        self.freq[number] += 1

    def find(self, value: int) -> bool:
        for num in self.freq:
            target = value - num
            if target in self.freq and (num != target or self.freq[num] > 1):
                return True
        return False

```

## ✓ 8. [641. Design Circular Deque] (Rewritten for full use of `collections.deque` )

```

from collections import deque

class MyCircularDeque:
    def __init__(self, k: int):
        self.k = k
        self.q = deque()

    def insertFront(self, value: int) -> bool:
        if len(self.q) < self.k:
            self.q.appendleft(value)
            return True
        return False

    def insertLast(self, value: int) -> bool:
        if len(self.q) < self.k:
            self.q.append(value)
            return True
        return False

    def deleteFront(self) -> bool:
        if self.q:
            self.q.popleft()
            return True
        return False

    def deleteLast(self) -> bool:
        if self.q:
            self.q.pop()
            return True
        return False

    def getFront(self) -> int:
        return self.q[0] if self.q else -1

    def getRear(self) -> int:
        return self.q[-1] if self.q else -1

    def isEmpty(self) -> bool:
        return not self.q

    def isFull(self) -> bool:
        return len(self.q) == self.k

```

## 9. 362. Design Hit Counter

```

from collections import deque

class HitCounter:
    def __init__(self):
        self.q = deque()

    def hit(self, timestamp: int) -> None:
        self.q.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        while self.q and self.q[0] <= timestamp - 300:
            self.q.popleft()
        return len(self.q)

```

## ✓ 10. 703. Kth Largest Element in a Stream

```

import heapq

class KthLargest:
    def __init__(self, k: int, nums: list[int]):
        self.k = k
        self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val: int) -> int:
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]

```